BACKEND API WITH FASTAPI

Prepared By: **Mahalakshmi**

Name: Mahalakshmi

Project Topic: Backend API Development using FastAPI

FastAPI is a modern Python web framework used to build fast, reliable and developer-friendly APIs. This document explains the basic concepts, architecture, implementation, testing and deployment of a backend API using FastAPI.

1. Introduction to Backend API

A backend API acts as a communication layer between a client application and a server. It receives requests, processes business logic, communicates with databases or other services, and returns responses.

FastAPI is designed for building APIs with Python. It supports asynchronous programming, automatic validation, interactive API documentation and type hints.

A typical flow is: Client → API endpoint → Business logic → Database/service → JSON response.

2. Prerequisites

Before creating a FastAPI backend, basic knowledge of Python, HTTP methods, JSON and REST API concepts is useful.

Required software includes Python 3.x, a code editor such as Visual Studio Code, and a terminal or command prompt. A virtual environment is recommended for dependency management.

Common packages are FastAPI and Uvicorn. Uvicorn is an ASGI server used to run the FastAPI application.

3. FastAPI Project Structure

A simple project can contain a main application file, route modules, schemas, models and configuration files.

Example structure: app/main.py, app/routes/, app/schemas/, app/models/, and requirements.txt.

Separating routes, schemas and business logic makes the project easier to maintain and expand.

4. Creating API Endpoints

An endpoint is a URL through which a client accesses a backend function. FastAPI provides decorators such as @app.get(), @app.post(), @app.put() and @app.delete().

GET is generally used to retrieve data, POST to create data, PUT/PATCH to update data, and DELETE to remove data.

FastAPI converts Python return values into JSON-compatible HTTP responses automatically.

5. Request Validation and Pydantic

FastAPI uses Python type hints and Pydantic models to validate incoming request data.

For example, a user registration model may contain name, email and password fields. If the submitted data does not match the expected format, FastAPI can return a validation error response.

Validation improves reliability because incorrect data can be rejected before it reaches application logic.

6. Database Integration

A real backend often needs persistent storage. FastAPI can work with relational and NoSQL databases through suitable Python libraries.

A common architecture separates database models from API schemas. The API receives validated input, performs database operations, and sends a structured response.

Database sessions, connection management and error handling should be designed carefully for production applications.

7. Authentication and Security

Authentication verifies who a user is, while authorization determines what that user is allowed to access.

FastAPI can be integrated with OAuth2, bearer tokens and JWT-based authentication systems. Passwords should never be stored as plain text; secure password hashing should be used.

Other important practices include input validation, HTTPS in production, secure secrets management, rate limiting where appropriate, and careful handling of errors.

8. Testing and API Documentation

FastAPI automatically provides interactive API documentation through OpenAPI. Developers can use the documentation interface to inspect endpoints and test requests.

Automated tests can verify successful responses, validation failures, authentication rules and database-related behavior.

Testing helps identify errors early and makes future changes safer.

9. Running and Deploying FastAPI

During development, Uvicorn can run the FastAPI application locally. A typical command is: uvicorn app.main:app --reload.

For deployment, the application can be hosted on a cloud or server environment using an ASGI server setup. Environment variables should be used for configuration and secrets.

Monitoring, logging, backups and HTTPS should be considered when moving an API into production.

10. Conclusion

FastAPI provides a practical way to create modern backend APIs with Python. Its type hints, validation, asynchronous support and automatic documentation make development efficient.

A well-structured FastAPI project can serve web applications, mobile applications and other client systems. Good architecture, validation, authentication, testing and deployment practices are important for building a dependable backend.

Prepared for: Mahalakshmi